

INTRODUCTION TO MACHINE LEARNING

Generalization and Feature Engineering

Interpolation, feature maps, the bias-variance trade-off, validation and regularization, with two worked examples from the history of statistics.

Summary: the recipe so far

1. Assume a linear hypothesis

$$h_{\theta}(x) = \theta^{\top} x = \sum_{i=0}^d \theta_i x_i$$

2. Cost function ↓

$$J(\theta) = \frac{1}{2} \sum_{i=1}^d \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

3. Minimize

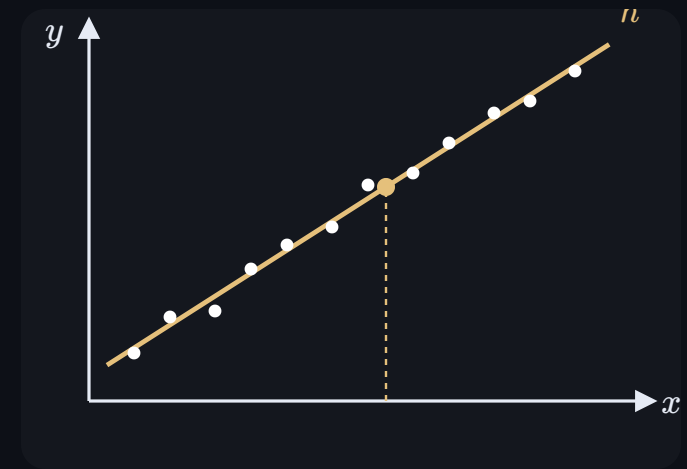
$$\hat{\theta} = \arg \min_{\theta} J(\theta)$$

5. Predict unseen data

$$y_{pred} = h_{\hat{\theta}}(x_{new})$$

4. Optimal predictor ↑

$$y = h_{\hat{\theta}}(x)$$



SGD

for $t = 1 \dots T$:

for all examples i :

$$\vec{\theta} := \vec{\theta} - \alpha \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) \vec{x}^{(i)}$$

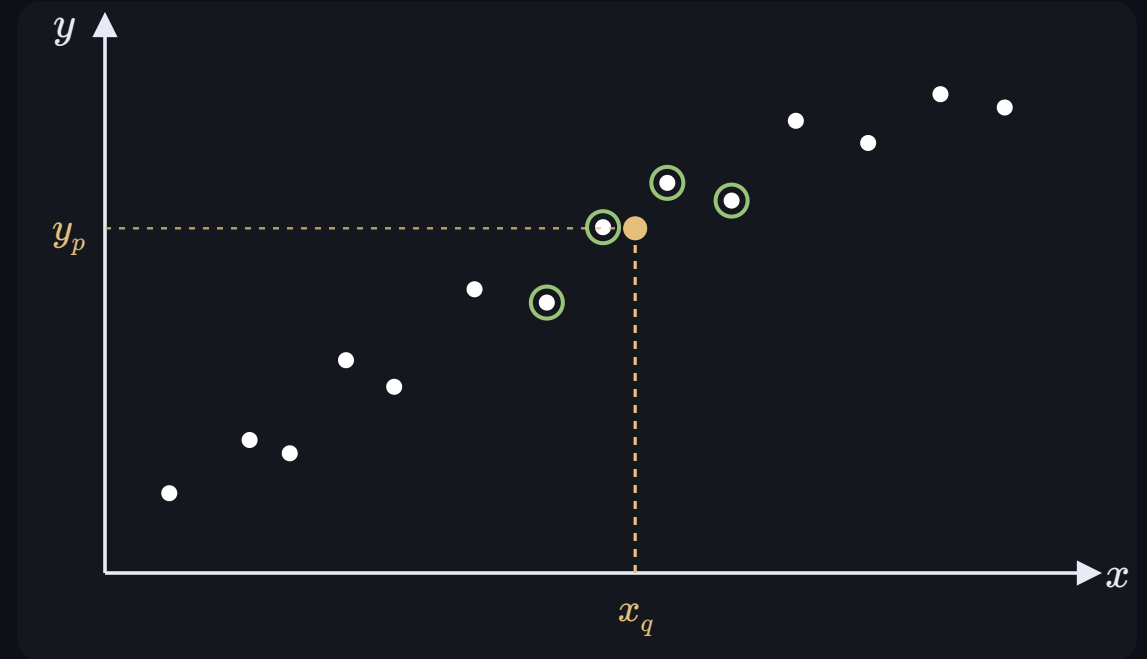
Everything after this lecture changes one line of that recipe: which h you are allowed to assume.

Given a new input, what's the output?

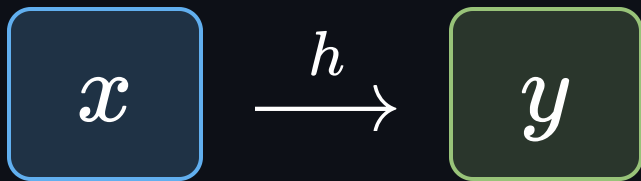
Input (size)	Output (price)
$x^{(1)}$	$y^{(1)}$
$x^{(2)}$	$y^{(2)}$
$x^{(3)}$	$y^{(3)}$
$x^{(4)}$	$y^{(4)}$
$\vdots$	$\vdots$

Query x_q goes in, **prediction** y_p comes out.

Weighted average of surrounding points: k-nearest neighbours

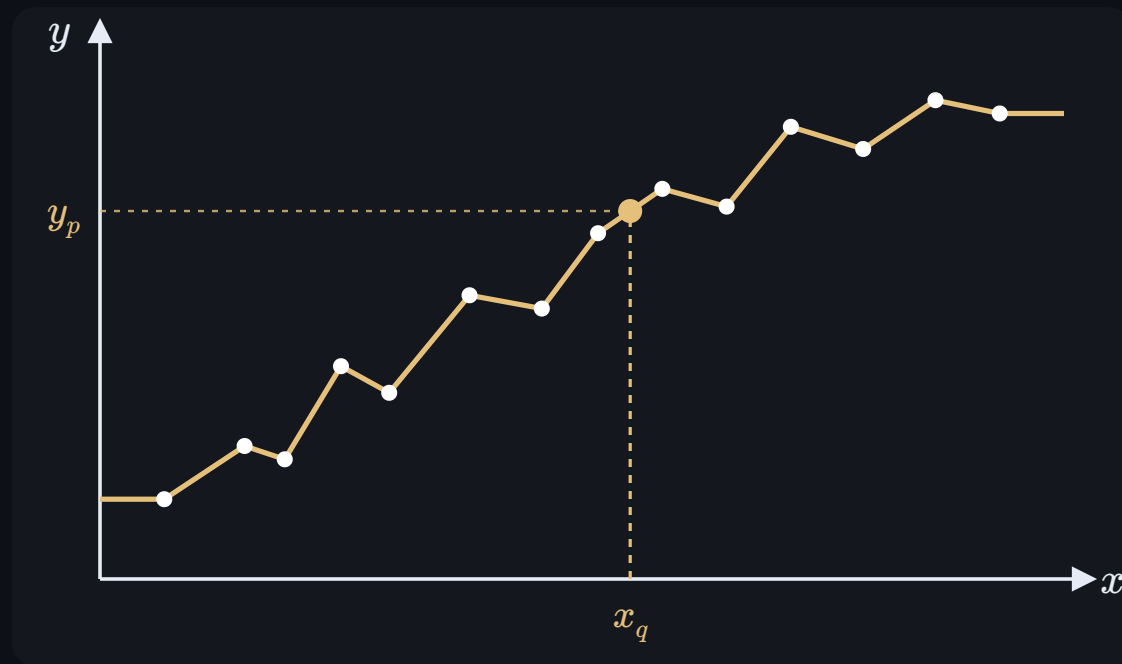


Given new input, what's the output?

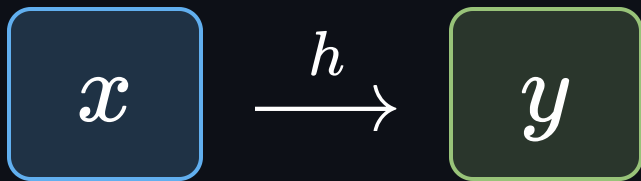


Given the data, find a function h , a.k.a. a **hypothesis**, that predicts outputs given inputs. $h : \mathbb{R} \rightarrow \mathbb{R}$

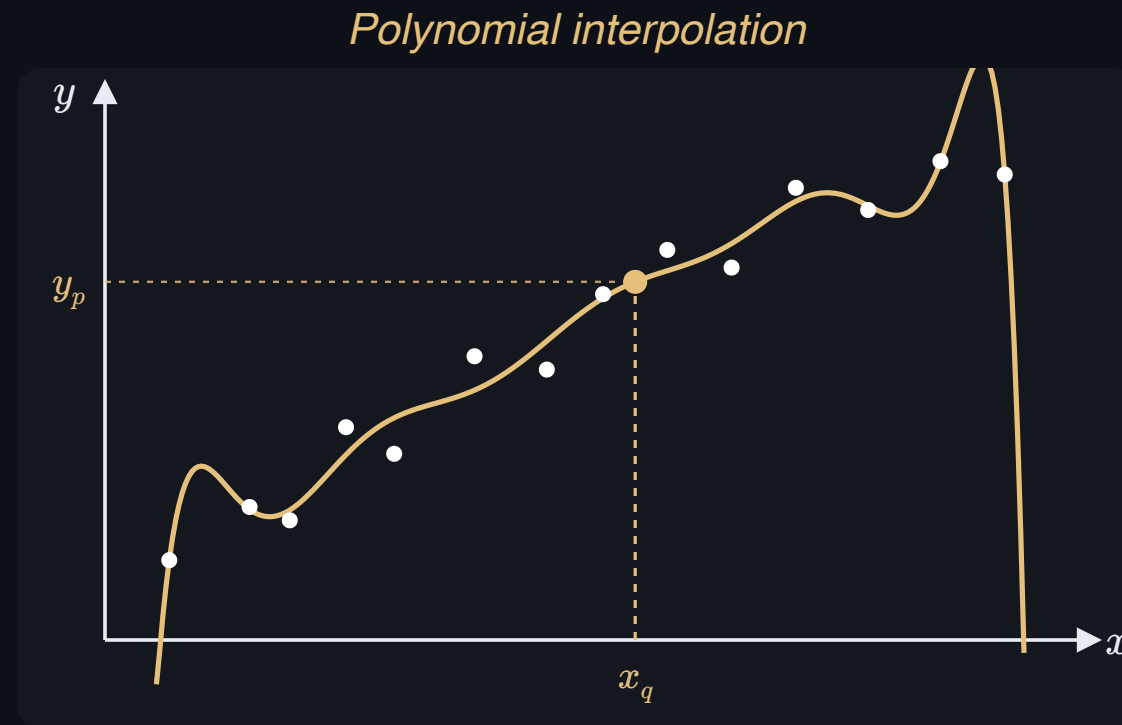
Linear interpolation



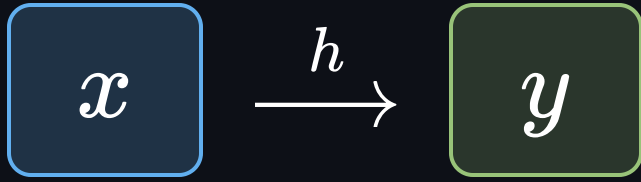
Given new input, what's the output?



Given the data, find a function h , a.k.a. a **hypothesis**, that predicts outputs given inputs. $h : \mathbb{R} \rightarrow \mathbb{R}$



How do you choose the hypothesis?



Given the data, find a function h , a.k.a. a **hypothesis**, that predicts outputs given inputs. $h : \mathbb{R} \rightarrow \mathbb{R}$

The same data admits all of them. Which one do you want?



A Feature Built in 1885

Francis Galton measured 928 grown children and their parents, and had to invent a feature before he could plot anything.

$$\text{mid-parent} = \frac{\text{father} + 1.08 \times \text{mother}}{2}$$

Two parents, two numbers, one axis. The 1.08 scales mothers up before averaging, so that a tall mother and a tall father count the same. It is a modelling decision, made before any fitting, and it is baked into every number that follows.

THIS IS $\Phi(X)$

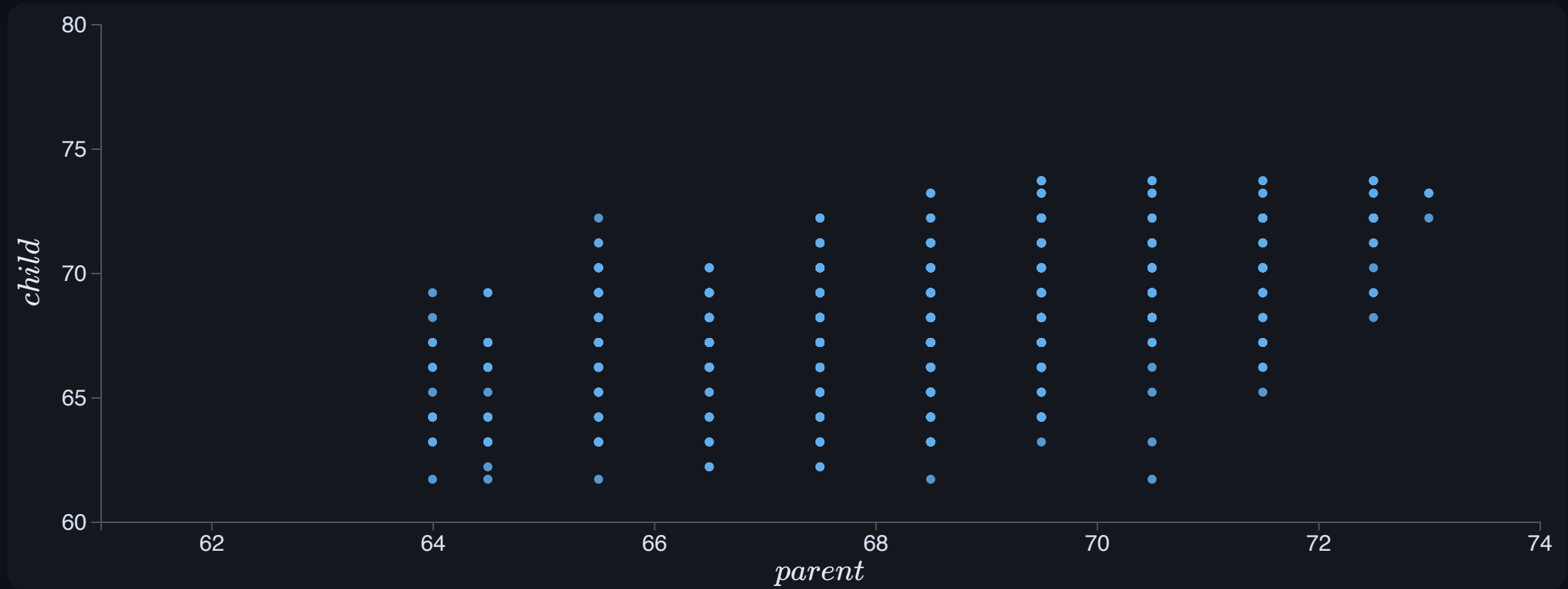
Two raw measurements became one engineered feature. The choice of 1.08 is an assumption you inherit whether or not you notice it.

inches father	mother	mid-parent	child
78.5	67.0	75.4	73.2
78.5	67.0	75.4	69.2
78.5	67.0	75.4	69.0
78.5	67.0	75.4	69.0
75.5	66.5	73.7	73.5

Five families from the original table. 934 rows, 205 families.

Count the Points

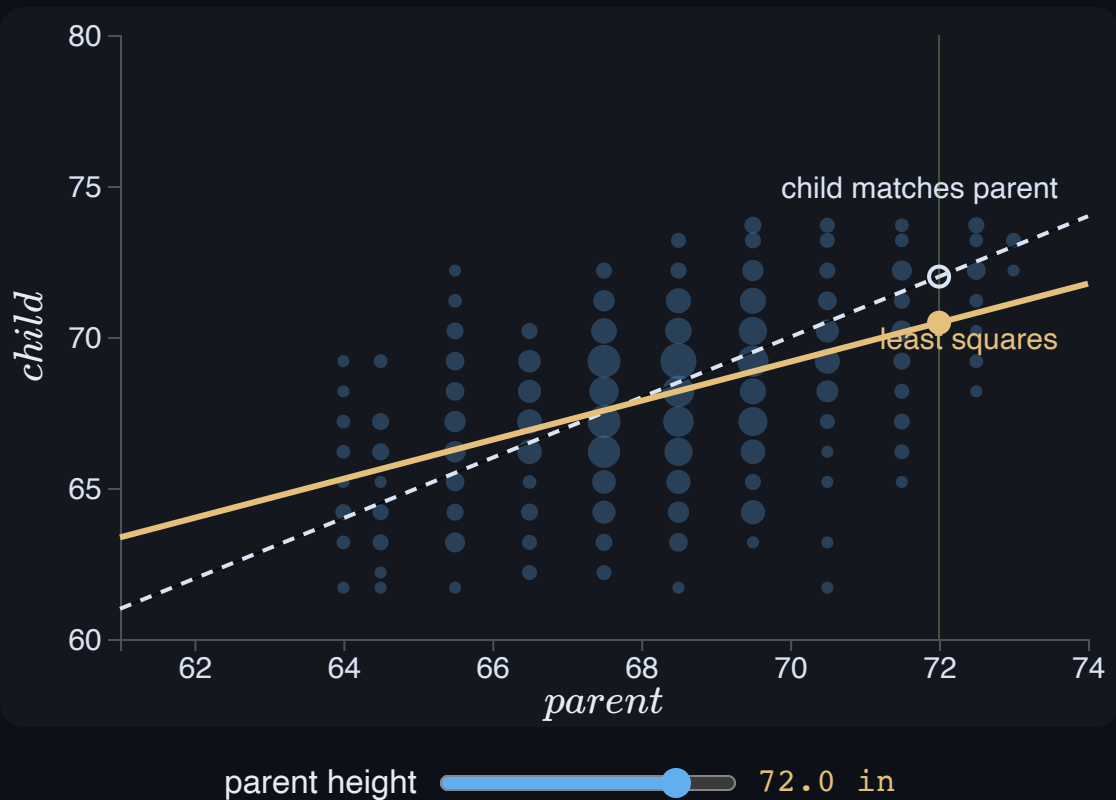
There are 928 children in this plot. Nothing about the plot says so.



As plotted Jittered Sized by count looks like about 100 points

Galton rounded every height, so 928 measurements land on 102 positions. One position holds 48 children. Three quarters of the data sits under a dot that hides at least nine others.

Where the Word Comes From



The fitted line is flatter than the dashed line where a child simply matches the parent. That gap is the whole story.

WHAT THE SLIDER SAYS

A parent at **72.0** in is **3.7** in above the parent average.
The fit predicts a child at **70.5** in, only **2.4** in above the child average.
Pulled back by 1.3 in towards the middle.

Galton called this *regression towards mediocrity*. The field is named after this slope, in this dataset, being less than one.

Tall parents do have tall children. Just less tall. Short parents likewise, in the other direction.

A Fifth of the Variance, and a Whole Discipline

THE FIT

$$\text{child} = 0.646 \times \text{parent} + 23.9$$

Correlation $r = 0.459$. Its square is the share of the variance in the child's height that the fit accounts for: residual over total, subtracted from one.

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} = 0.21$$

BOTH THINGS ARE TRUE

The parents remove a fifth of the uncertainty and leave four fifths. Real enough to name a field, and a poor predictor of any one child. A low R^2 is not a failed model, nor a high one a good one.

typical error in a child height

knowing nothing



2.52 in

knowing the parents



2.24 in

The parents buy you 11 per cent off the spread, which is what $R^2 = 0.21$ means in inches.

Spread of child heights, before and after the parents are taken into account.

A Model With a Date On It

von Foerster, Mora and Amiot, *Science*, 1960. Title: "Doomsday: Friday, 13 November, A.D. 2026".

They fitted world population with one curve:

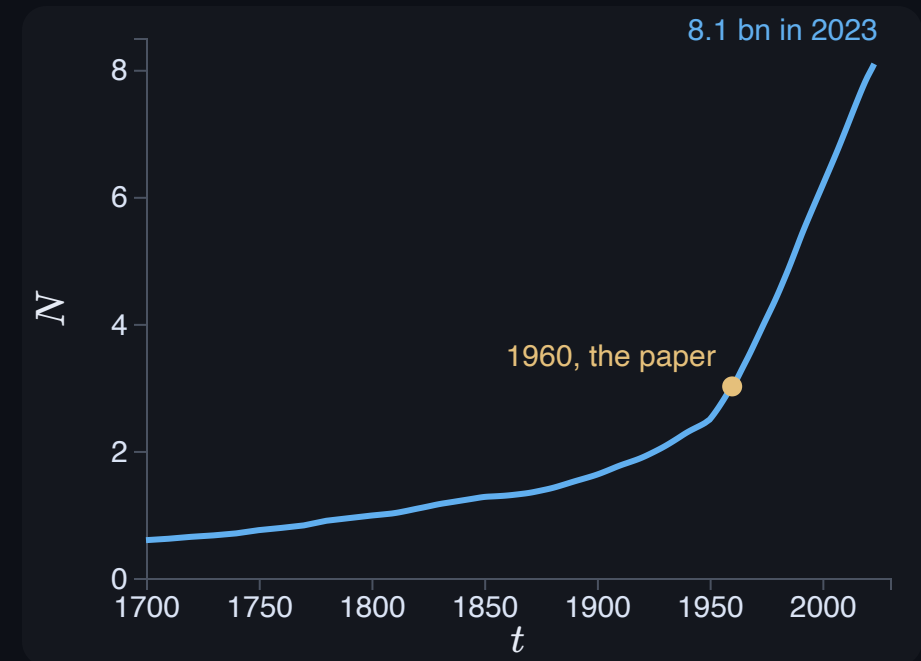
$$N(t) = \frac{C}{(t_0 - t)^k}$$

Fitted on two thousand years of data, they got $t_0 = 2026.87$ and $k = 0.99$. The curve goes to infinity at t_0 , and that date is a Friday.

EIGHT WEEKS FROM THIS LECTURE

13 November 2026 falls in this semester, three weeks before our last class.

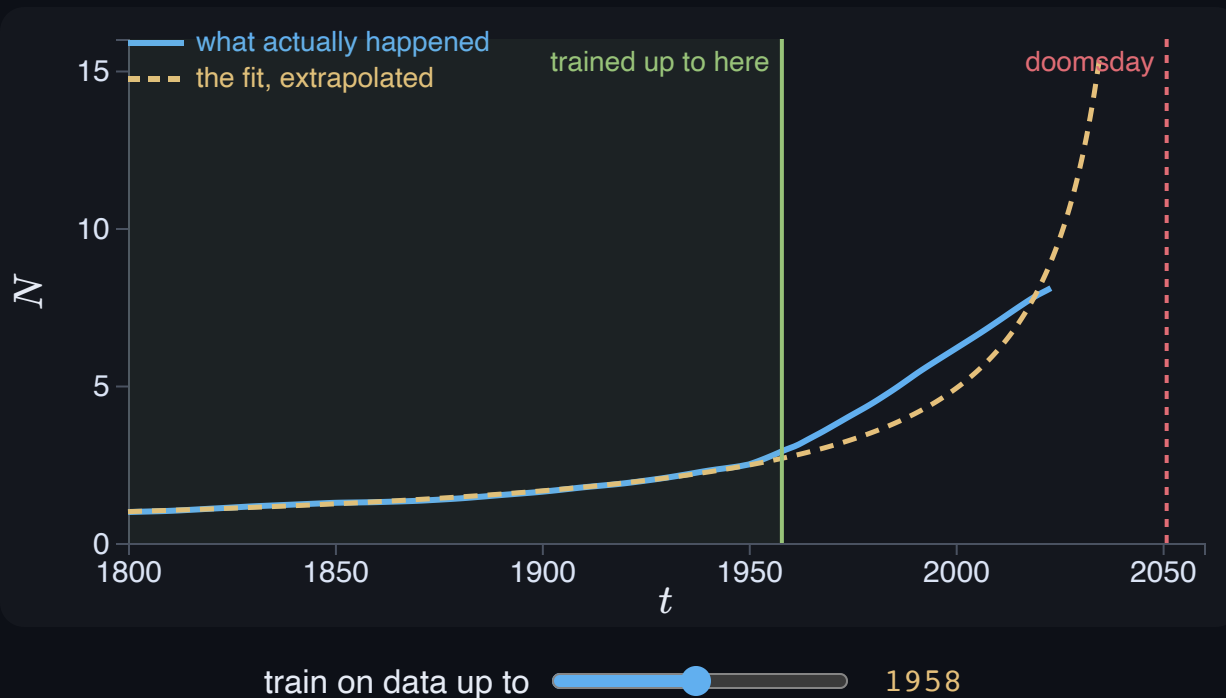
To be fair to them: the paper was partly tongue in cheek. They were not predicting infinite people. They were arguing that the growth pattern of the previous two millennia was about to break. It did.



World population, 1700 to 2023. [Our World in Data](#), vendored CSV.

Fit It Yourself, Then Look Forward

Set $k = 1$ and the model is a straight line in $1/N$, so this is one least squares fit.



WHAT THE FIT SAYS

Trained on 159 years, 1800 to 1958.

In sample $R^2 = 0.9935$.

Population goes to infinity in **2051**.

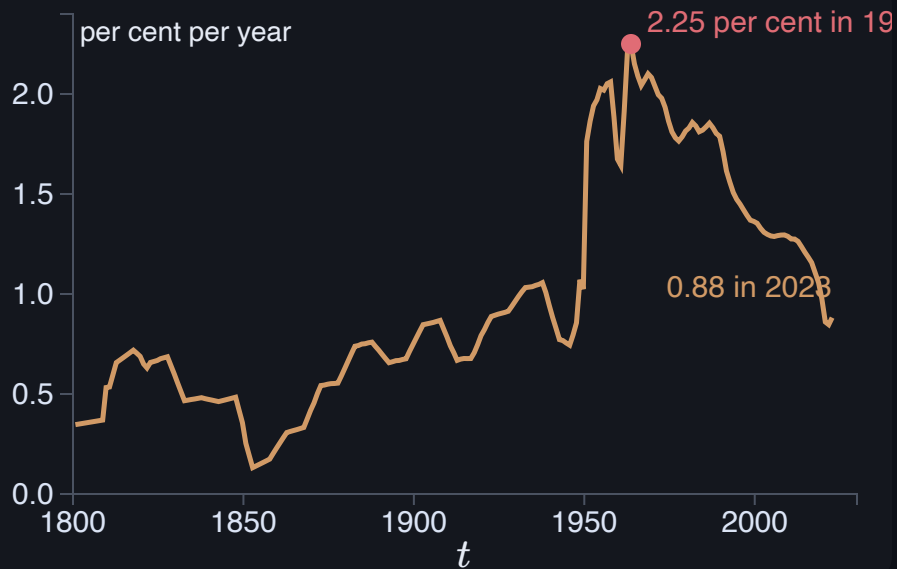
It says 2023 holds 8.9 bn. There were 8.1 bn, so it is off by **10 per cent**.

Drag the cutoff and doomsday wanders from 2063 to 2039. It never admits it is wrong: R^2 stays above 0.99 throughout, *highest* at 2020, while the 2023 forecast drifts from 8.4 billion past 14.

WHY NONE OF THEM SAYS 2026

This fit uses $k = 1$ from 1800; von Foerster used $k = 0.99$ back to 1 AD on 1960 figures. Each choice moves the date: 2051, 2000, or the published 2026.87.

What a Curve Cannot Know



Annual growth rate of world population.

Growth peaked at **2.2 per cent** a year in the early 1960s and has fallen ever since: 1.36 in 2000, 0.88 in 2023. Births per woman fell as child mortality fell, incomes rose and cities grew.

THE MECHANISM WAS NEVER IN THE MODEL

N as a function of t alone has nowhere to put a demographic transition. The curve could not bend, because nothing in it bends.

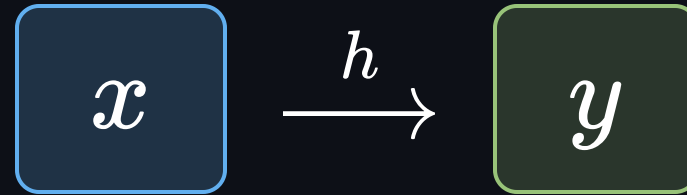
Two ways a fit misleads you

Galton: $R^2 = 0.21$, and the finding is real.

von Foerster: $R^2 = 0.99$, and the model is wrong.

Read the original: [Doomsday, Science 132 \(1960\)](#). A modern critique: [arXiv 0910.3056](#), and [the end of hyperbolic growth](#).

Assume a linear hypothesis



$$h_{\theta}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \dots$$

$$h_{\theta}(\mathbf{x}) = \underbrace{[\theta_0, \theta_1, \theta_2, \theta_3, \dots]}_{\text{weights } \theta} \cdot \underbrace{[1, x_1, x_2, x_3, \dots]}_{\mathbf{X} \text{ inputs}}$$

$$h_{\theta}(\mathbf{x}) = \theta \cdot \mathbf{x} = \theta^{\top} \mathbf{x}, \text{ with } \theta \in \mathbb{R}^{d+1} \text{ and } \mathbf{x} \in \mathbb{R}^{d+1}$$

Assume a linear hypothesis

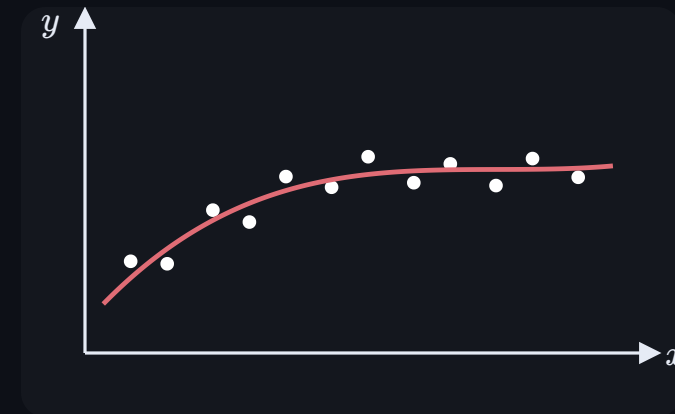
h does **not** have to be linear in x : build $\phi(x)$ and be linear in that.

Example: construct a polynomial model

$$h_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \dots$$

$$h_{\theta}(x) = \underbrace{[\theta_0, \theta_1, \theta_2, \theta_3, \dots]}_{\theta} \cdot \underbrace{[1, x, x^2, x^3, \dots]}_{\phi(x)}$$

$$h_{\theta}(x) = \theta^{\top} \phi(x): \text{linear in } \theta, \text{ not in } x$$



A curve, from a model that is linear in its parameters

Assume a linear hypothesis

Written out, the feature map is just a list of functions of x , and the model is a weighted sum of them.

$$h_{\theta}(x) = \theta^{\top} \phi(x) = \theta_0 \phi_0(x) + \theta_1 \phi_1(x) + \theta_2 \phi_2(x) + \dots$$

- **Each ϕ_j is a choice.** Powers of x give a polynomial. Sines give a Fourier series. Indicator functions give a step model.
- **The shapes.**

$$\phi(x) \in \mathbb{R}^p, \quad \theta \in \mathbb{R}^p, \quad h_{\theta}(x) \in \mathbb{R}$$

- **Fitting is unchanged.** Least squares on $\phi(x)$ instead of on x . Every formula you already have still applies.

Example: construct a polynomial model

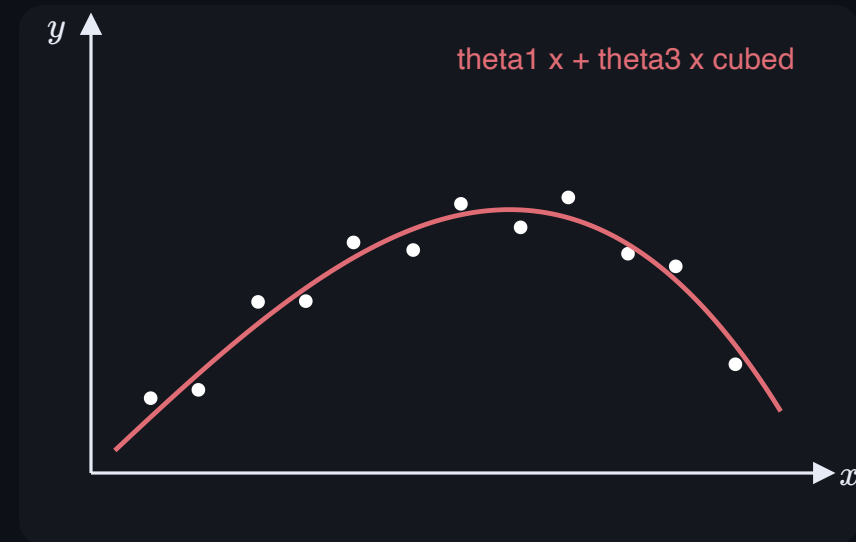
Pick two features and nothing else:

$$h_{\theta}(x) = \theta_1 x + \theta_3 x^3$$

$$h_{\theta}(x) = [\theta_1, \theta_3] \cdot [x, x^3]$$

$\phi(x) = [x, x^3]$, the feature map

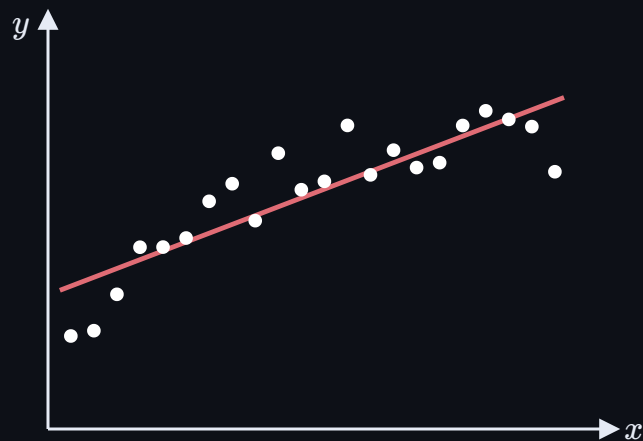
No constant term and no x^2 : those are modelling choices, and they are choices you make before any fitting happens.



How to choose $\phi(x)$? How to optimize over $\phi(x)$

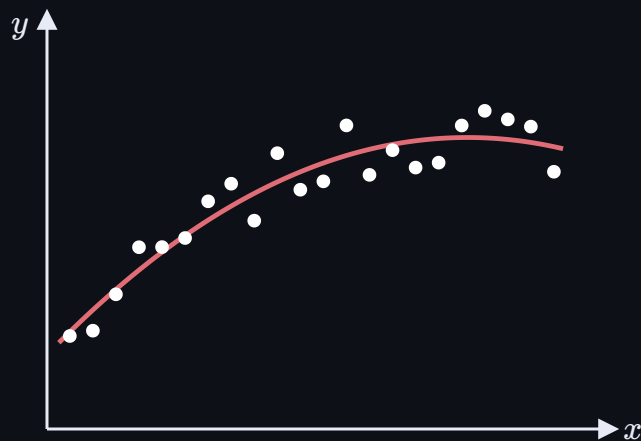
Underfitting

High Bias



$$\phi(x) = [1, x]$$

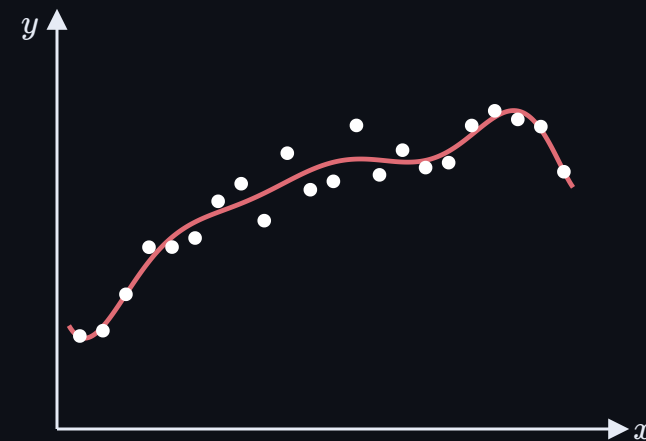
Just right



$$\phi(x) = [1, x, x^2]$$

Overfitting

High Variance



$$\phi(x) = [1, x, x^2, x^3, \dots]$$

Same data, same fitting procedure. The only thing that changed is which features you allowed.

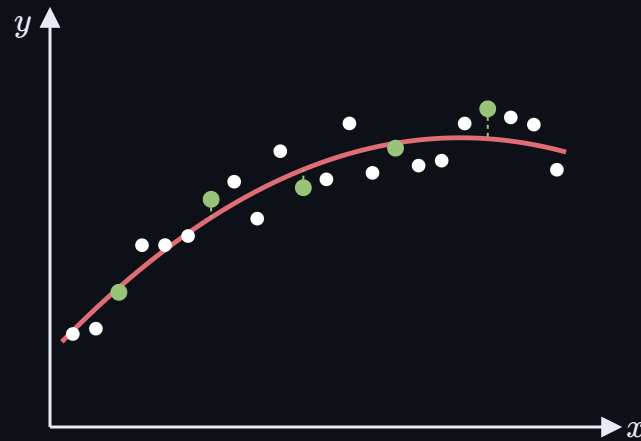
How can we tell if $\phi(\cdot)$ is good?

The purpose of machine learning is to **generalize to unseen data**

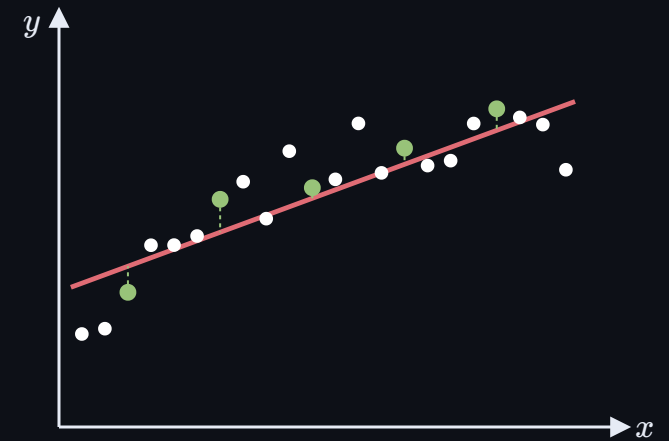
Hold out set



Small loss on the test set



Large loss on the test set



Create a **test set** to evaluate the model

$$\phi(x) = [1, x, x^2]$$

$$\phi(x) = [1, x]$$

The green points are the same points in all three panels. They were never used to fit anything.

How can we tell if $\phi(\cdot)$ is good?

Define objective functions for each subset



Create a **test set** to evaluate the model

Split data:

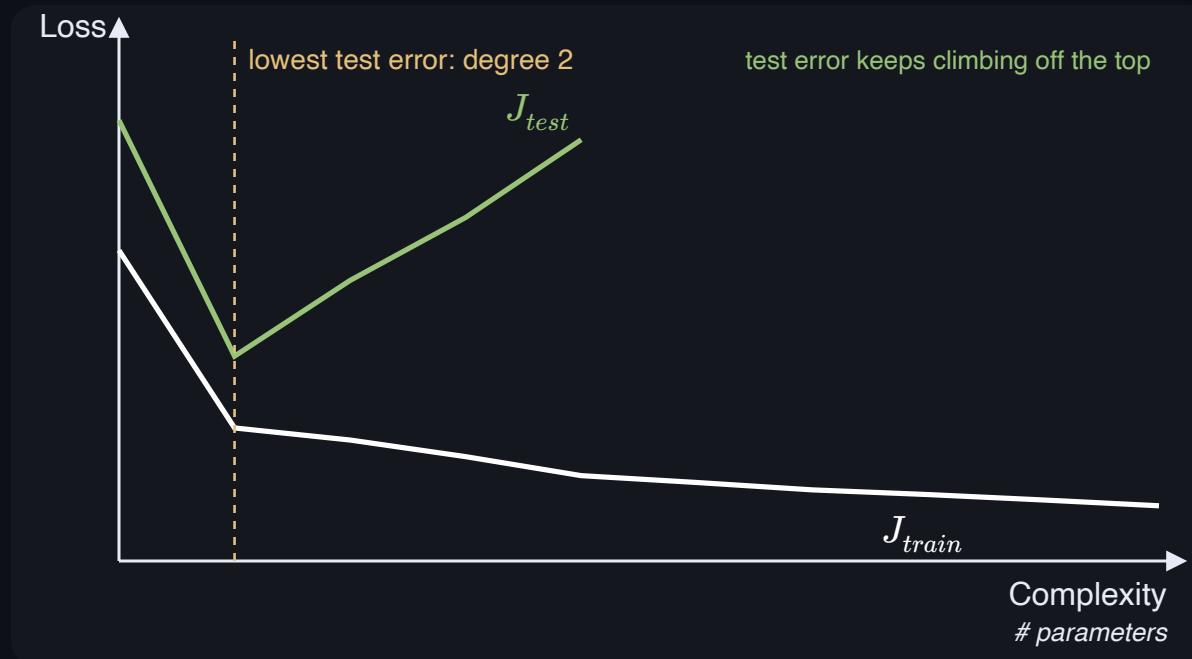


$$J_{train}(\theta) = \frac{1}{2d_{tr}} \sum_{i=1}^{d_{tr}} \left(\theta^\top \phi(x^{(i)}) - y^{(i)} \right)^2$$

$$J_{test}(\theta) = \frac{1}{2d_{tt}} \sum_{i=1}^{d_{tt}} \left(\theta^\top \phi(x^{(i)}) - y^{(i)} \right)^2$$

Variance Bias Trade-off

Error as a function of complexity



- **Training error falls** as you add features, and keeps falling. It is not evidence of anything.
- **Test error turns around.** Everything to the left of the turn is bias, everything to the right is variance.
- **You cannot see the turn** from the training set alone. That is what the held-out data is for.

Both curves are measured, not drawn: each point is a least squares fit at that degree on the same data.

Other Hyperparameters

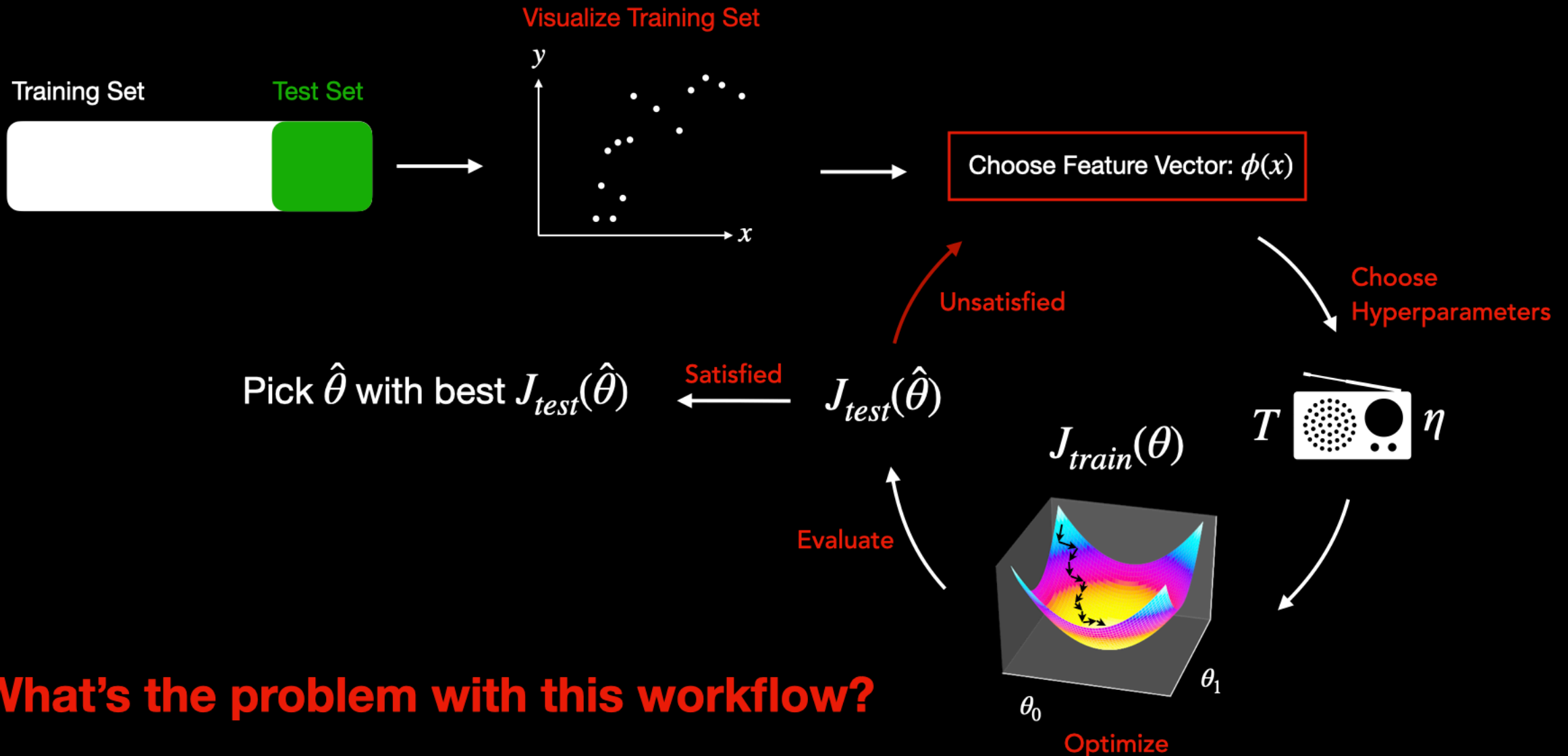
ϕ is not the only unknown parameter over which we want to optimize

- T : number of epochs, how long you train for.
- η : step size, how far each update moves.
- ϕ : the feature vector, which functions of x the model is allowed to use.

WHY THIS MATTERS

None of these is fitted by least squares. They sit outside the optimization, which is exactly why they need their own held-out data to choose between them.

Machine Learning Workflow



What's the problem with this workflow?

Optimize over ϕ and other hyperparameters

Define objective functions for each subset



Test set: evaluate model at the end of hyperparameter optimization

Validation set: evaluation model during hyperparameter optimization

Split data:

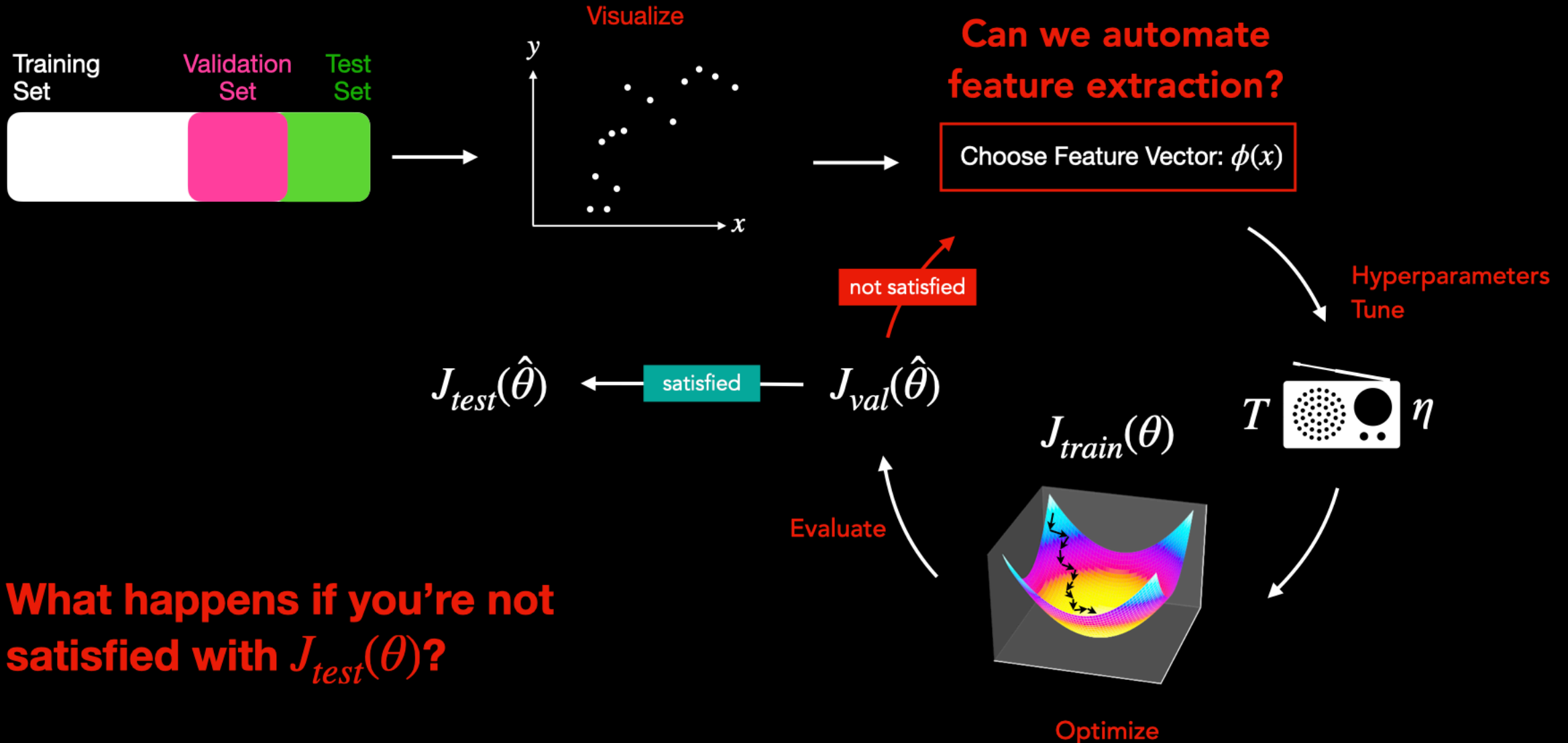


$$J_{train}(\theta) = \frac{1}{2d_{tr}} \sum_{i=1}^{d_{tr}} (\theta^T \phi(x^{(i)}) - y^{(i)})^2$$

$$J_{val}(\theta) = \frac{1}{2d_{val}} \sum_{i=1}^{d_{val}} (\theta^T \phi(x^{(i)}) - y^{(i)})^2$$

$$J_{test}(\theta) = \frac{1}{2d_{tt}} \sum_{i=1}^{d_{tt}} (\theta^T \phi(x^{(i)}) - y^{(i)})^2$$

Machine Learning workflow - Cross Validation



Remedies to Overfitting

Practical tips to decrease overfitting

- Make the model simpler if it's overfitting, and more complex if it's underfitting
 - **Recursive Feature Elimination:** start with all features and drop them one by one while tracking the loss
 - Get rid of features that you think are irrelevant in predicting the desired output
- Add a **regularization term** to the cost function:

$$J_{reg}(\theta) = J(\theta) + \lambda R(\theta)$$

Regularization

Force fitting parameters to be smaller - 'shrink' hypothesis class

$$h_{\theta}(x) = 100.2 + 50.6x + 70.4x^2 + 1345x^3 + 200.3x^4$$

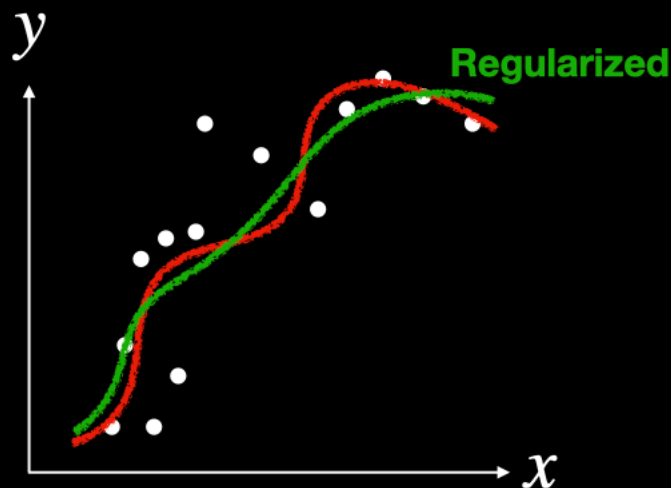
$$J_{reg}(\theta) = J(\theta) + \lambda R(\theta)$$

L1 Regularization

$$R(\theta) = \|\theta\|_1$$

$$h_{\theta}(x) = 5.1x + 7.2x^2 + 3.3x^4$$

Less coefficients



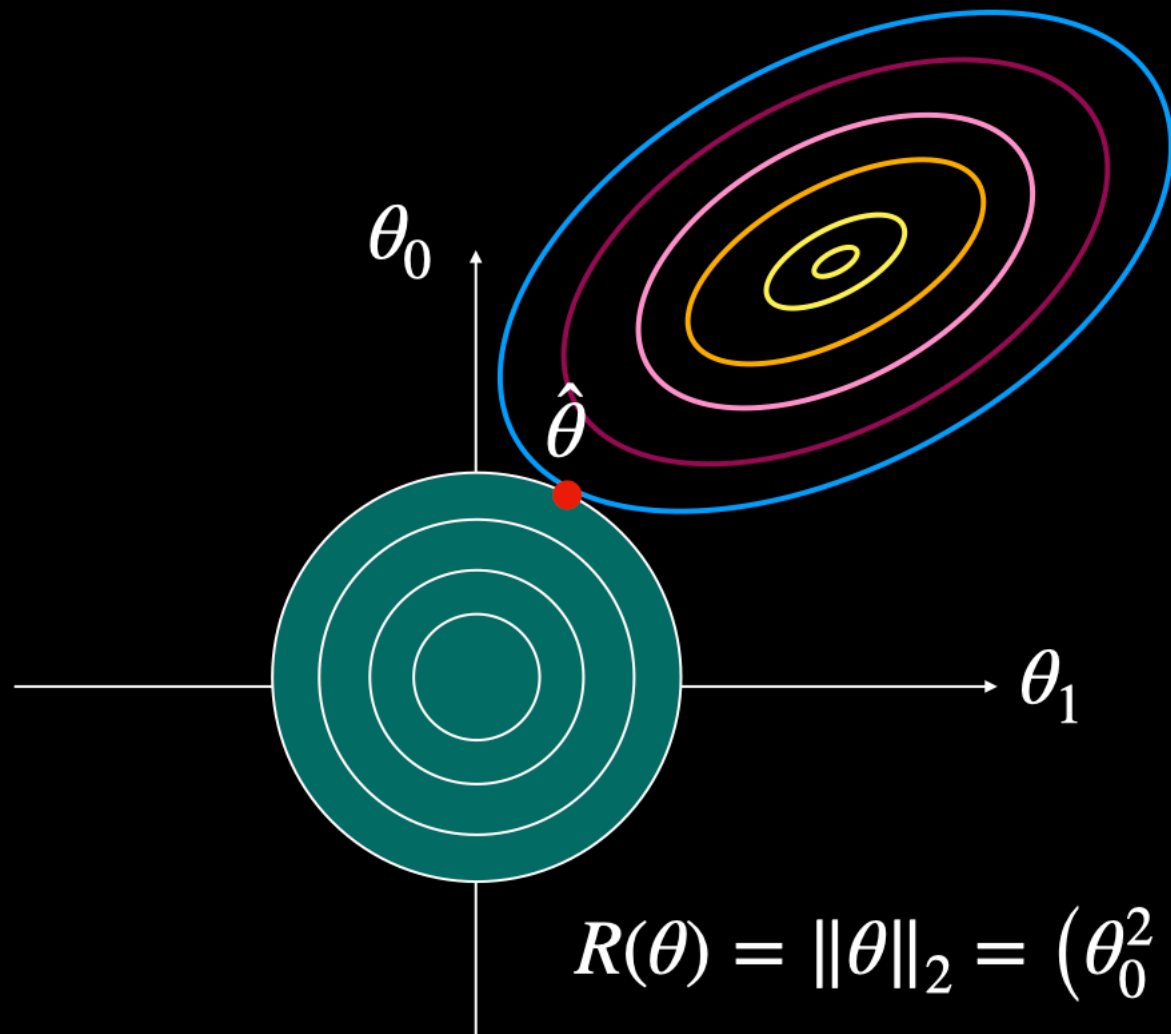
L2 Regularization

$$R(\theta) = \|\theta\|_2$$

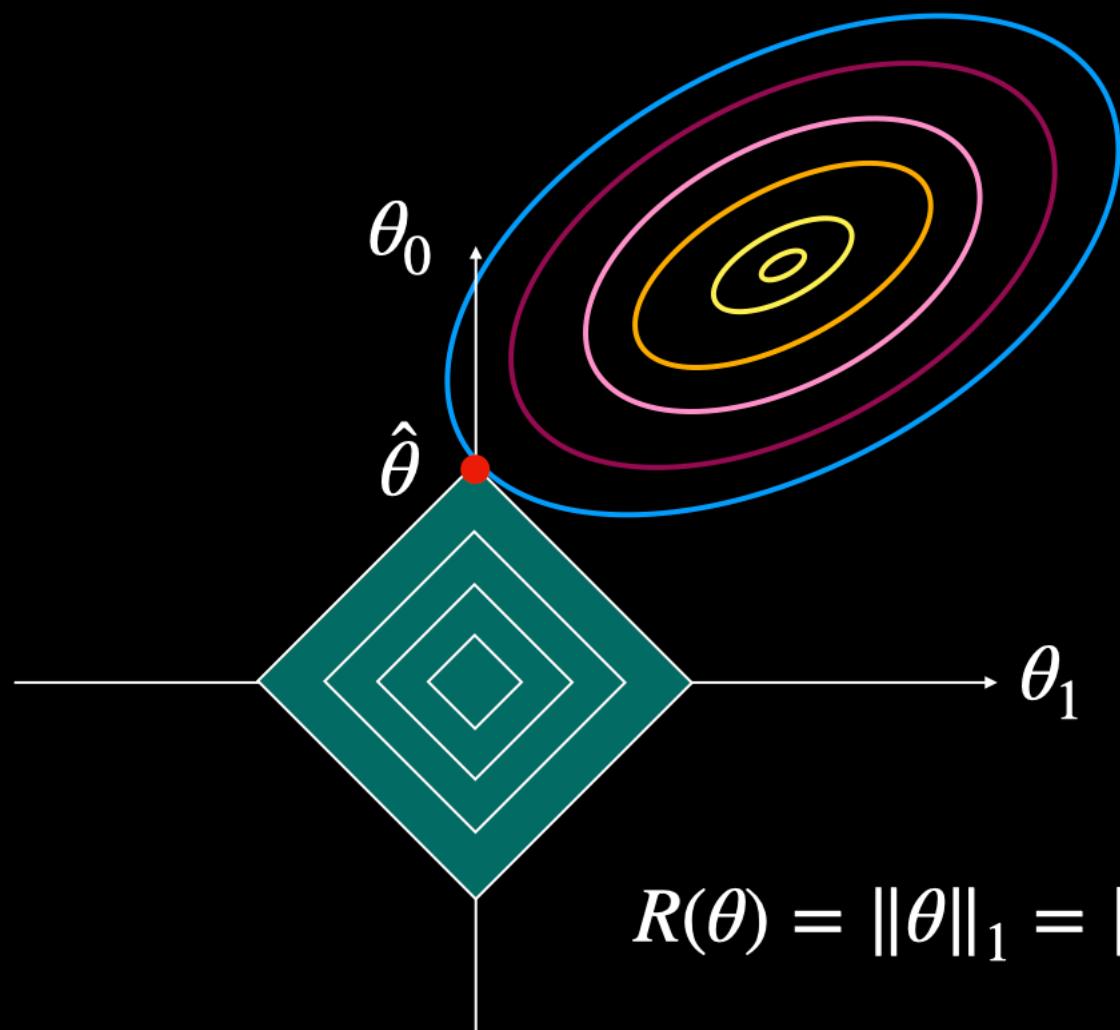
$$h_{\theta}(x) = .1 + 5.2x + 7.4x^2 + .05x^3 + 2.3x^4$$

Smaller coefficients

L2 Regularization



L1 Regularization



$$R(\theta) = \|\theta\|_1 = |\theta_0| + |\theta_1| + \dots$$

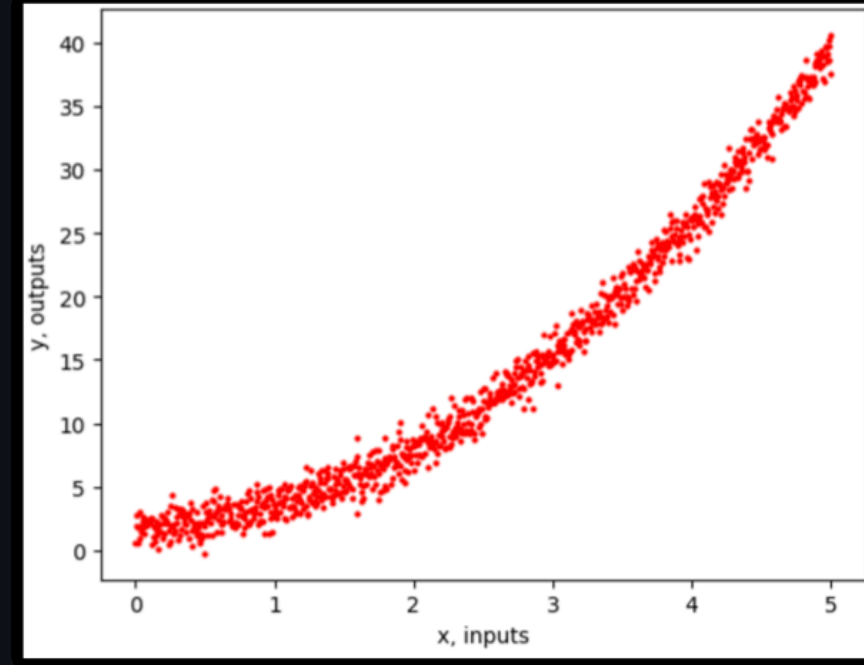
Polynomial Regression, from Scratch

Everything from the last slides, in code: build features, split, fit by gradient descent, and check that it generalizes.

Create synthetic data

```
# synthetic parameters
num_points = 1000
var = 1
a = 1.5
b = 2

# generate data
x = np.linspace(0, 5, num_points)
y = 1.5 * x**2 + b + var * np.random.normal(0, 1, num_points)
```



Feature engineering

```
# Create features

def design_matrix(x, degree):
    X = np.zeros((len(x), degree+1))
    for i in range(X.shape[1]):
        X[:, i] = x**i
    return X

degree = 2
X = design_matrix(x, degree)
y = y.reshape(-1, 1)
```

Shuffle and split

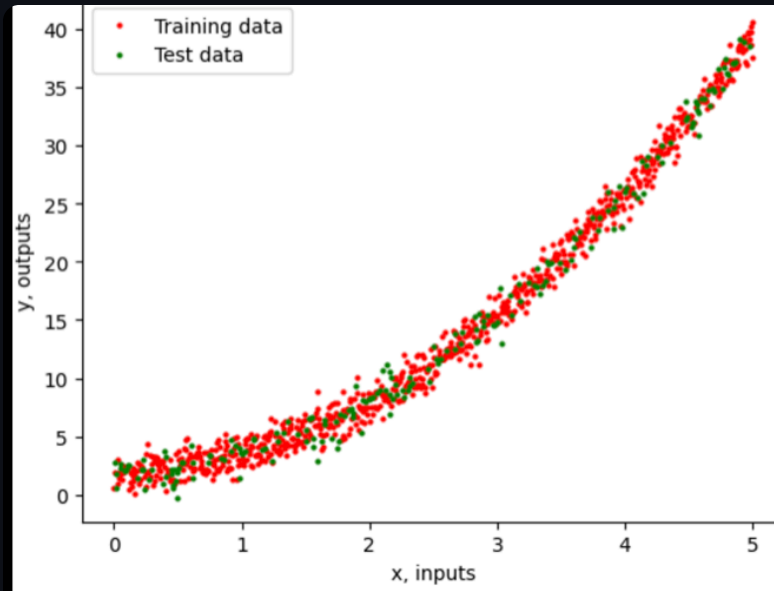
```
# Split data
n_train = int(0.8 * num_points)
n_test = num_points - n_train

shuff_index = np.random.permutation(num_points)
X_shuffle = X[shuff_index]
y_shuffle = y[shuff_index]

X_train = X_shuffle[:n_train]
X_test = X_shuffle[n_train:]
y_train = y_shuffle[:n_train]
y_test = y_shuffle[n_train:]
```

Visualize the split

```
# Plot training data  
plt.plot(X_train[:, 1], y_train, 'ro', ms=2, label='Training data')  
plt.plot(X_test[:, 1], y_test, 'go', ms=2, label='Test data')  
plt.legend(); plt.show()
```



Cost function and gradient descent

COST FUNCTION

```
def cost_function(X, y, theta):  
    m = len(y)  
    return 1/(2*m) * np.sum((X @ theta - y)**2)
```

RUN IT

```
theta = np.random.randn(degree+1, 1)  
learning_rate = 0.01  
num_iters = 10000  
theta, J_history = gradient_descent(  
    X_train, y_train, theta,  
    learning_rate, num_iters)
```

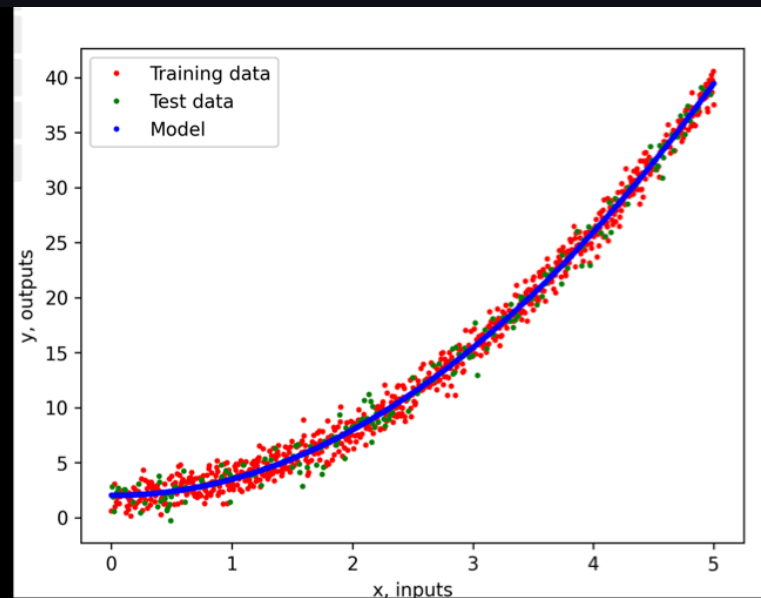
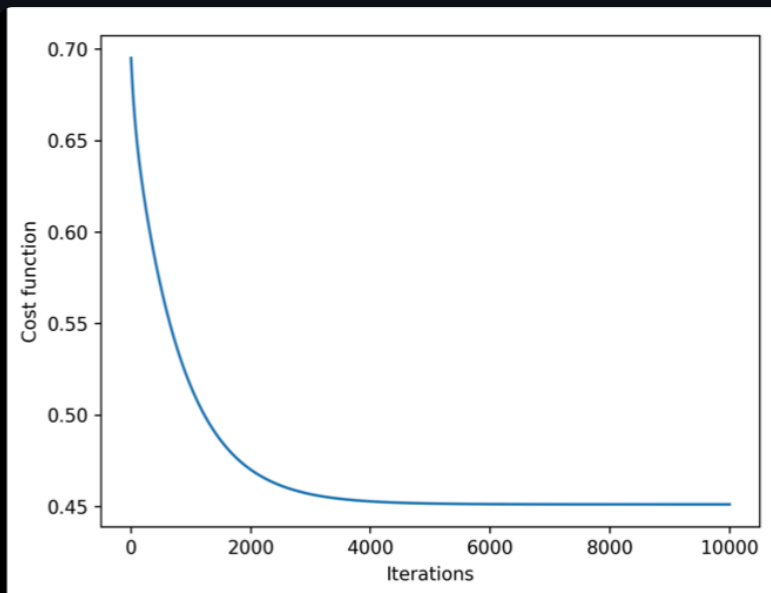
```
theta → array([[ 2.042], [-0.025], [ 1.504]])
```

GRADIENT DESCENT

```
def gradient_descent(X, y, theta,  
                    learning_rate, num_iters):  
  
    m = len(y)  
    J_history = np.zeros(num_iters)  
    for i in range(num_iters):  
        theta = theta - (learning_rate/m) \  
            * X.T @ (X @ theta - y)  
        J_history[i] = cost_function(X, y, theta)  
    return theta, J_history
```

Plot the result

```
# plot comparison
plt.plot(X_train[:, 1], y_train, 'ro', ms=2, label='Training data')
plt.plot(X_test[:, 1], y_test, 'go', ms=2, label='Test data')
plt.plot(X_train[:, 1], X_train @ theta, 'bo', ms=2, label='Model')
plt.legend(); plt.show()
```



Train and test loss

```
test_loss = cost_function(X_test, y_test, theta)
train_loss = cost_function(X_train, y_train, theta)
```

```
print(f'Test loss: {test_loss}')
print(f'Train loss: {train_loss}')
```

```
Test loss: 0.508970692715051
Train loss: 0.4511700483353495
```

WHY THIS MATTERS

Test and train loss are close, so the degree-2 fit generalizes: it does about as well on data it never saw as on data it trained on.